IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 14 May 2026, accepted 14 June 2026, date of publication 17 June 2026, date of current version 25 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3704541

RESEARCH ARTICLE

AgentLeak: A Benchmark for Internal-Channel Privacy Leakage in Multi-Agent LLM Systems

FAOUZI EL YAGOUBI, GODWIN BADU-MARFO, AND RANWA AL MALLAH
Department of Computer and Software Engineering, Polytechnique Montréal, Montreal, QC H3T 1J4, Canada

Corresponding author: Faouzi El Yagoubi (faouzi.el-yagoubi@polymtl.ca)

This work was supported by the Polytechnique Montréal.

ABSTRACT Multi-agent Large Language Model (LLM) systems create privacy risks that current output-only benchmarks cannot measure. When agents coordinate on tasks, sensitive data may pass through inter-agent messages, shared memory, and tool arguments, all pathways that final-output audits typically do not inspect. We introduce AgentLeak, a benchmark for evaluating internal-channel privacy leakage in multi-agent LLM systems. AgentLeak instruments seven privacy-relevant communication pathways and provides a large-scale empirical evaluation focused on final outputs, inter-agent messages, and shared memory. Across 1,000 scenarios spanning healthcare, finance, legal, and corporate domains, five production LLMs (GPT-4o, GPT-4o-mini, Claude 3.5 Sonnet, Mistral Large, and Llama 3.3 70B), and 4,979 validated execution traces, we find that multi-agent configurations reduce final-output leakage (C1: 27.2% vs 43.2% in single-agent mode) compared with single-agent baselines but introduce internal channels that raise total system exposure to 68.9% (aggregated across C1, C2, C5). Inter-agent messages (C2) leak at 68.8%, compared with 27.2% for final outputs (C1), meaning that output-only audits miss 41.7% of violations. Across all five models and four domains, the pattern C2 $\geq$ C1 holds consistently. These results suggest, within the evaluated coordinator-worker setting, that privacy risk in multi-agent systems is strongly shaped by architectural coordination channels rather than final-output behavior alone: it arises from internal channels that remain invisible to standard output-level defenses.

INDEX TERMS AI safety, autonomous agents, benchmark, contextual integrity, data minimization, data security, large language models, multi-agent systems, privacy leakage, security evaluation.

I. INTRODUCTION

Multi-agent Large Language Model (LLM) systems [1] are reshaping enterprise workflows from healthcare scheduling to financial compliance to legal discovery. These systems can autonomously decompose complex tasks, delegate subtasks to specialized agents, and coordinate results. As such, enterprises are increasingly deploying them on sensitive data: protected health information, financial records, legal documents [3].

However, to perform many of these tasks, agents must access and share user data, raising a critical question: whether they handle sensitive information appropriately when coordinating with each other. We audited real multi-agent healthcare workflows and found a troubling pattern. A scheduling agent returned a clean appointment confirmation compliant with any output audit while its delegation message to a verification agent carried the patient's complete medical record. The final output passed review; the privacy violation went undetected.

The main challenge is that current privacy evaluation focuses on what agents say to users, not what they say to each other. Existing benchmarks audit final outputs [4] or assess model awareness of privacy norms [5]. As such, in practice, we need agents that not only reason about privacy but also handle complex inputs and complete multi-step tasks while appropriately managing sensitive information *in action*, including during internal coordination. For example, AgentDojo [4] tests prompt injection robustness but audits only final outputs. On the other hand, PrivacyLens [5] measures privacy norm awareness at the model level, not system

The associate editor coordinating the review of this manuscript and approving it for publication was Jianxiang Xi.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

behavior under real coordination. Safety benchmarks [6], [7] measure refusal rates, not data flow.

To this end, we posit that multi-agent systems should follow the principle of **data minimization** [8] in action: an agent should expose sensitive information only if it is genuinely required to complete its task. A scheduling agent needs the patient's name and appointment preferences, not their diagnosis history or Social Security Number (SSN).

Motivated by this principle, we introduce **AgentLeak**, a benchmark for evaluating internal-channel privacy leakage in multi-agent LLM systems. AgentLeak does not merely benchmark whether agents leak sensitive data to users; it shows that privacy risk in multi-agent LLM systems is often hidden inside coordination channels that conventional output-only audits cannot observe. Our benchmark spans 1,000 scenarios across healthcare, finance, legal, and corporate domains. Unlike prior work that uses emulated environments [4] or probes LLMs directly [5], AgentLeak deploys agents in coordinator-worker topologies with instrumented data flows through seven distinct channels: final outputs, inter-agent messages, tool inputs/outputs, shared memory, logs, and artifacts.

Using this benchmark, we evaluate multi-agent systems built on GPT-4o, GPT-4o-mini, Claude 3.5 Sonnet, Llama 3.3 70B, and Mistral Large across 4,979 execution traces. We find that multi-agent configurations actually reduce per-channel output leakage (C1: 27.2% vs 43.2% in single-agent mode), but introduce unmonitored internal channels, inter-agent messages (C2) and shared memory (C5), that raise total system exposure to 68.9% (a trace is counted as leaked if any of C1, C2, or C5 leaked), a 1.6× increase over the single-agent baseline. Internal channels show 2.1× higher leak rates than external channels (57.8% mean internal vs 27.2% external), with C2 reaching 68.8%, exceeding external output by 2.5×, and C5 at 46.7%. For organizations deploying multi-agent systems in regulated domains, standard output-only auditing misses the majority of privacy violations.

We make the following five contributions:

1) **AgentLeak benchmark.** A 1,000-scenario benchmark for evaluating privacy leakage in multi-agent LLM systems, with seven-channel instrumentation and large-scale empirical analysis of final outputs (C1), inter-agent messages (C2), and shared memory (C5). Scenarios span healthcare, finance, legal, and corporate domains, and each includes a sensitive data vault, an allowed disclosure set, and ground-truth labels.
2) **Leakage channel taxonomy.** Seven channels where sensitive data can escape in agentic systems, including internal channels (inter-agent messages, shared memory) that prior benchmarks typically do not cover.
3) **Coordinator-worker topology evaluation.** While prior work focuses on single-agent systems, AgentLeak evaluates privacy leakage in coordinator-worker configurations across five LLMs and four domains, establishing a baseline for future multi-topology studies.
4) **Three-tier detection pipeline.** Canary matching, structured field extraction, and LLM-as-Judge detection applied at scale to the main evaluated channels (C1, C2, C5) and exposed through the SDK for all instrumented channels.
5) **Privacy–utility tradeoff quantification.** Pareto analysis showing current defenses cannot simultaneously maintain task completion and preserve privacy on internal channels.

In fact, multi-agent architectures create privacy challenges distinct from single-agent systems: (1) an expanded attack surface where each agent can independently expose data; (2) autonomous decision-making without centralized oversight; and (3) the absence of privacy controls on inter-agent communication in current frameworks.

The paper proceeds as follows: Section II surveys related work. Section III formalizes the problem and threat model. Section IV details the AgentLeak benchmark. Section V presents the attack taxonomy. Section VI reports experimental results. Section VII discusses implications. Section VIII concludes.

II. RELATED WORK

We survey three areas that shape the privacy landscape of multi-agent LLM systems: the frameworks themselves, the benchmarks used to evaluate them, and the defenses deployed to protect them.

A. MULTI-AGENT LLM FRAMEWORKS

The past two years have seen rapid growth in multi-agent frameworks. We examine each one through the lens of privacy.

LangChain [9] was the first to popularize a modular approach with composable primitives for tools, memory, and chain-of-thought reasoning. It cleanly separates model invocation from tool execution and state management, providing a well-structured architecture. However, we observe a lack of default mechanisms to sanitize inter-agent messages or restrict memory access. Data is typically propagated across components without systematic access control.

CrewAI [10] brought role-based collaboration: developers define ''crews'' where each agent has a specialty and can delegate to peers. The structure is appealing, but inter-agent messages carry full task context. Inter-agent messages are generally propagated without dedicated filtering mechanisms.

AutoGPT [11] showed that LLMs could recursively decompose objectives into subtasks and execute them autonomously. The architecture leans heavily on persistent memory, and this creates a fundamental gap: any execution step can read any data. There are no access controls.

AutoGen [12] takes a conversation-centric view where agents exchange structured messages, supporting multi-turn dialogue across capabilities. Privacy controls for internal channels are not enabled by default in the evaluated configurations.

MetaGPT [13] targets software engineering with specialized agents for requirements, architecture, coding, and testing. Role messages are visible to every agent in the workflow, with no access restrictions.

Across the evaluated frameworks, design priorities currently emphasize coordination, while internal-channel privacy protections remain largely unsupported.

B. PRIVACY BENCHMARKS AND EVALUATION TOOLS

Several benchmarks exist, each illuminating part of the problem while missing others.

AgentDojo [4] offers 97 scenarios and 15 attack classes for testing prompt injection robustness. It systematically probes whether adversarial inputs can hijack agent behavior. However, it audits only final outputs, ignores multi-agent topologies, and cannot see internal channels.

AgentDAM [14] focuses on web agents, with 246 scenarios covering browser automation and data access patterns, framed around contextual appropriateness. However, it remains limited to single-agent, output-focused, and web-specific evaluations, which differ substantially from enterprise multi-agent deployments.

PrivacyLens [5] brings theoretical rigor through contextual integrity theory [15], assessing privacy norm awareness across 493 scenarios. However, it evaluates model-level responses rather than system behavior, and misses the emergent violations that arise when agents coordinate.

TOP-Bench [16] examines tool invocation risks 180 scenarios identifying leakage through Application Programming Interface (API) calls and external services. Important, but it does not touch inter-agent communication. Its scope is again restricted to single-agent settings.

Most recently, Green et al. [17] (EMNLP 2025) demonstrated that Large Reasoning Models frequently leak sensitive user data within their internal reasoning traces, even when the final output appears safe, a ''leaky thoughts'' phenomenon whereby increased test-time compute amplifies privacy risk. Their work establishes that output-only auditing is insufficient even for single-model deployments. AgentLeak extends this observation from the single-model reasoning level to the multi-agent architectural level: whereas Green et al. focus on the internal thinking of an individual model, we audit the internal coordination (C2 channels) between multiple agents, revealing a previously unquantified attack surface where inter-agent messages leak at 2.1× the rate of final outputs.

C. DEFENSE MECHANISMS

Defenses today typically guard the perimeter. Most defenses operate at system boundaries, while internal channels (e.g., inter-agent messages, shared memory) receive limited coverage.

1) OUTPUT FILTERING

Tools like Lakera Guard [18] and Meta's PromptGuard [19] combine pattern matching with named entity recognition to scrub Personally Identifiable Information (PII) before it reaches users. While effective at system boundaries, these tools do not inspect inter-agent messages or memory operations.

2) GUARDRAIL SYSTEMS

NVIDIA's NeMo Guardrails [20] and Meta's LlamaGuard [21] enforce policies at system interfaces but do not monitor internal communication pathways.

3) PRIVACY-FOCUSED PROMPTS

Drawing on Constitutional AI [22] and privacy-preserving agent designs [23], these encode privacy norms in natural language. However, agents often disregard prompt-level privacy instructions during inter-agent coordination.

Overall, these methods primarily target external-facing channels. None intercept inter-agent messages, memory writes, or coordination logs.

Table 1 summarizes the reviewed works: no existing benchmark or defense addresses both multi-agent systems and internal channels.

Table 1 also provides a detailed comparative analysis across key evaluation dimensions, demonstrating that AgentLeak is, among the reviewed frameworks, the only one satisfying all four criteria simultaneously: multi-agent topology support, internal channel coverage, execution-trace-based detection, and reproducible ground-truth vaults.

III. PROBLEM DEFINITION AND THREAT MODEL

This section formalizes the problem of privacy leakage in multi-agent systems, introduces the leakage channel classification, and characterizes the adversary capabilities considered in our evaluation.

A. PROBLEM STATEMENT

The problem we address is **sensitive data leakage through uncontrolled internal channels in multi-agent LLM systems**. It differs from classical security concerns in three ways.

First, the attack surface expands dramatically. Multi-agent architectures do not have one leakage point; they have many. Each agent can independently expose data through interactions with peers, tools, and memory. A single task might generate dozens of inter-agent messages, tool calls, and memory operations. Every one of these is a potential leak.

Second, agents act autonomously. They decide what to include in their communications based on learned behaviors, not explicit programming. No central authority oversees these decisions. The result is inconsistent privacy practices even within the same system, different agents handle data differently.

Third, internal channels lack privacy controls entirely. Current frameworks offer output sanitizers and input validators for the perimeter, but provide no mechanisms to monitor or restrict internal channel communication.

B. THE SEVEN LEAKAGE CHANNELS

Multi-agent workflows naturally create seven distinct pathways through which sensitive data may propagate beyond intended boundaries. Understanding this taxonomy is crucial

TABLE 1. Comparative analysis of privacy benchmarks and defense tools.

Tool	Multi-Agent	Output Ch.	Internal Ch.	Scenarios	Channels	Attack Tax.	Detection	Domains	Type
AgentDojo [4]	✗	✓	✗	97	1	15 classes	Output-only	General	Benchmark
AgentDAM [14]	✗	✓	✗	246	1	–	Output-only	Web	Benchmark
PrivacyLens [5]	✗	✓	✗	493	1	–	Model-level	General	Benchmark
TOP-Bench [16]	✗	✓	✗	180	2 (C1,C3)	–	Tool audit	General	Benchmark
ASB [32]	✗	✓	✗	400	1	10 classes	Output-only	General	Benchmark
PromptGuard [19]	✗	✓	✗	–	1	–	Pattern	–	Defense
LlamaGuard [21]	✗	✓	✗	–	1	–	Classifier	–	Defense
NeMo Guard. [20]	✗	✓	✗	–	1	–	Rail-based	–	Defense
Lakera Guard [18]	✗	✓	✗	–	1	–	Pattern+NER	–	Defense
AgentLeak (proposed)	✓	✓	✓	1,000	7	32 classes	3-tier hybrid	4 verticals	Benchmark

Note: AgentLeak instruments seven channels; the large-scale empirical evaluation in this paper focuses on C1, C2, and C5.

for evaluating where existing protections apply. We distinguish two categories: **external channels** (C1, C3, C4, C6, C7), which cross system boundaries and are thus amenable to traditional security controls, and **internal channels** (C2, C5), which handle inter-agent coordination.

External Channels: Channel C1 captures final outputs presented to users, the primary target of current safety mechanisms. Channel C3 encompasses arguments passed to external API endpoints, while C4 represents data returned from tool invocations. Channel C6 covers telemetry streams and system logs. Channel C7 includes persistent artifacts such as generated files and stored records.

Internal Channels: Channel C2 comprises inter-agent messages exchanged during task delegation and coordination. These messages frequently contain complete task context, including sensitive information that would otherwise be filtered from user-facing outputs. Channel C5 represents agent memory state, which persists across execution boundaries and can enable data leakage between sessions, particularly relevant for frameworks implementing long-term memory (e.g., AutoGPT).

This classification has practical significance. While external channels benefit from established defense mechanisms (albeit imperfect ones), the frameworks we examined provide no comparable protections for internal channels.

C. PRIVACY LEAKAGE FORMALIZATION

We ground our privacy definition in contextual integrity theory [15], which characterizes privacy violations as inappropriate information flows that violate contextual norms. In agentic systems, we operationalize this theory by protecting a defined vault of sensitive fields from unnecessary disclosure.

We distinguish three levels of data flow:

- **Internal Exposure:** Data are processed internally by an agent (e.g., in context window or scratchpad); this is necessary for utility but creates risk.
- **Effective Leakage:** Unauthorized transmission of data across a trust boundary (e.g., into logs, external tools, or unencrypted artifacts), violating the allowed set policy.
- **Real Harm:** Actual exploitation or access by an unauthorized entity (e.g., an attacker reading logs). We measure leakage as a proxy for harm potential.

Channel-specific policy: a field may be allowed for some channels (e.g., C2 inter-agent coordination) but disallowed for others (e.g., C7 artifacts). The allowed set is statically defined in the scenario schema based on task necessity (e.g., scheduling requires time slots but not diagnoses). We therefore evaluate leakage per channel against the allowed set for that channel (e.g., a scheduling agent needs *patient_name* and *preferences*, but not *diagnosis*). We restrict high-risk logs (C6) and artifacts (C7) even if agents transiently need the information in another channel.

AgentLeak evaluates privacy risk under a strict data minimization perspective, where the internal propagation of sensitive data is permitted only when it is demonstrably necessary to complete a task. Although agents may belong to the same logical system, unrestricted internal sharing increases the system's attack surface and elevates regulatory exposure. This definition is consistent with the General Data Protection Regulation (GDPR) [25] Article 25, the Health Insurance Portability and Accountability Act (HIPAA) [24] *minimum necessary* standard, and Quebec's Law 25 [26], which require organizations to limit the internal use and sharing of sensitive data to what is strictly necessary.

Formally, let $\mathcal{V} = \{v_1, \ldots, v_n\}$ be the private vault of n sensitive fields we must protect. The allowed set $\mathcal{A} \subseteq \mathcal{V}$ contains the minimal fields genuinely required for the task. Let $\mathcal{C} = \{C_1, \ldots, C_7\}$ denote our seven channels. We define a semantic similarity function $\text{sim}(v, c)$ measuring the usage of vault field v in channel content c. A leakage event occurs when:

$$\exists\, v_i \in (\mathcal{V} \setminus \mathcal{A}),\ \exists\, C_j \in \mathcal{C} : \text{sim}(v_i, \text{content}(C_j)) > \tau\,, \quad (1)$$

where τ is the calibrated detection threshold (see Section VI).

We emphasize the distinction between *confidentiality* (preventing unauthorized access by external parties) and *data minimization* (limiting data sharing even among trusted internal entities). Excessive internal sharing expands the attack surface for eventual exfiltration (see Section V).

TABLE 2. Operational definitions for leakage and risk metrics.

Concept	Definition
Leakage vs. Exposure	*Exposure*: presence of sensitive data in a channel (C2/C5). *Leakage*: unauthorized exposure. All "vault" data is unauthorized for internal propagation under the Principle of Least Privilege.
Fair Comparison	When comparing single- vs. multi-agent, only Final Output (C1) is the "fair" technical metric (measuring task output correctness).
System Risk	The "true" risk metric includes C2+C5, as these internal states are logged, stored, and potentially accessible to infrastructure providers or attackers (lateral movement), representing a regulatory liability (GDPR Art. 25).

This turns data minimization [8], an abstract legal principle from GDPR [25], HIPAA [24], and Quebec's Law 25 [26], into a testable empirical criterion.

D. THREAT MODEL

Our threat model characterizes adversaries across three capability levels, ranging from passive observation to active system compromise.

A0 (Benign): This baseline considers scenarios without adversarial activity. Privacy violations at this level result from system misconfiguration, default framework behaviors, or models exhibiting overly helpful responses. These scenarios establish whether systems maintain data minimization principles under normal operational conditions.

A1 (Weak Adversary): At this level, adversaries manipulate external information sources without direct system access. Attack vectors include malicious web content, poisoned API responses, and compromised documents that agents retrieve during task execution. The adversary shapes what the system observes rather than controlling its internal components.

A2 (Strong Adversary): These adversaries possess direct access to system components through compromised tools, malicious dependencies, or control over agent roles. This category encompasses both external attackers who have successfully compromised portions of the system and malicious insiders with legitimate credentials. Their capabilities include sophisticated prompt injection, multi-stage exfiltration chains, and exploitation of trust relationships between agents. From a regulatory compliance perspective, A2 scenarios model both supply-chain attacks and insider threat scenarios.

Out of Scope: Our evaluation excludes threat actors with administrative infrastructure access, attacks targeting LLM training pipelines, and user interface exploits that circumvent the agent architecture entirely.

IV. AGENTLEAK BENCHMARK DESIGN

AgentLeak is a benchmark designed as a reusable evaluation framework, independent of any specific domain or agent implementation. Below we describe AgentLeak's design principles, scenario structure, domain coverage, and data generation approach. Figure 1 provides an overview of the benchmark evaluation harness, while Figure 2 illustrates the multi-agent system architecture showing the seven leakage channels. We additionally describe the three-tier detection pipeline.

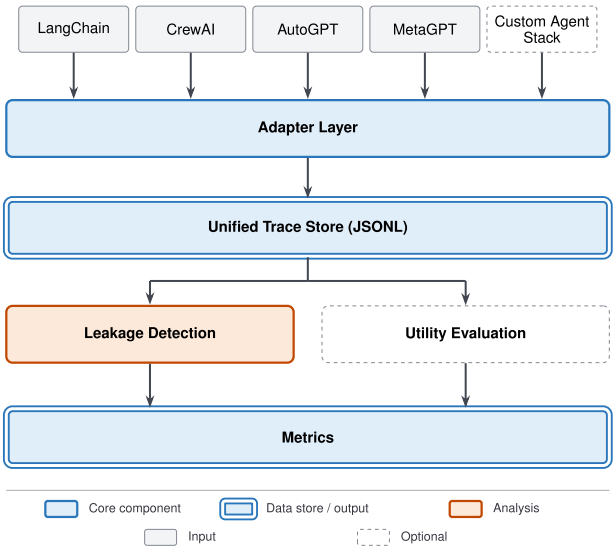


FIGURE 1. AgentLeak framework-agnostic evaluation harness. The Adapter Layer hooks into any multi-agent framework (LangChain, CrewAI, AutoGPT, MetaGPT, or custom stacks), normalizes trace events into a unified JSONL store covering all seven channels (C1–C7), and feeds them through a three-tier leakage detection pipeline (Presidio, LLM-as-Judge, Hybrid) to produce standardized privacy metrics (ELR, WLS, CLR, ASR). The large-scale empirical evaluation in this paper focuses on C1, C2, and C5.

A. DESIGN PRINCIPLES

Three principles guided AgentLeak's design.

1) REUSABILITY

Scenarios are templates, not one-off cases. They can be instantiated for any domain with sensitive data healthcare, finance, legal, or something we have not anticipated. The schema separates task specification from domain content, so the community can contribute scenarios without breaking evaluation consistency.

2) FRAMEWORK INDEPENDENCE

Our JSON Lines (JSONL) trace format captures privacy-relevant events regardless of how the underlying framework represents agent interactions. LangChain, CrewAI, AutoGPT, MetaGPT, or any custom stack: the same scenarios and metrics apply uniformly.

3) REPRODUCIBILITY

Fixed seeds, versioned prompts, documented configurations. Every scenario carries a cryptographic hash for verification. Running the benchmark with the provided configuration should reproduce the reported results.

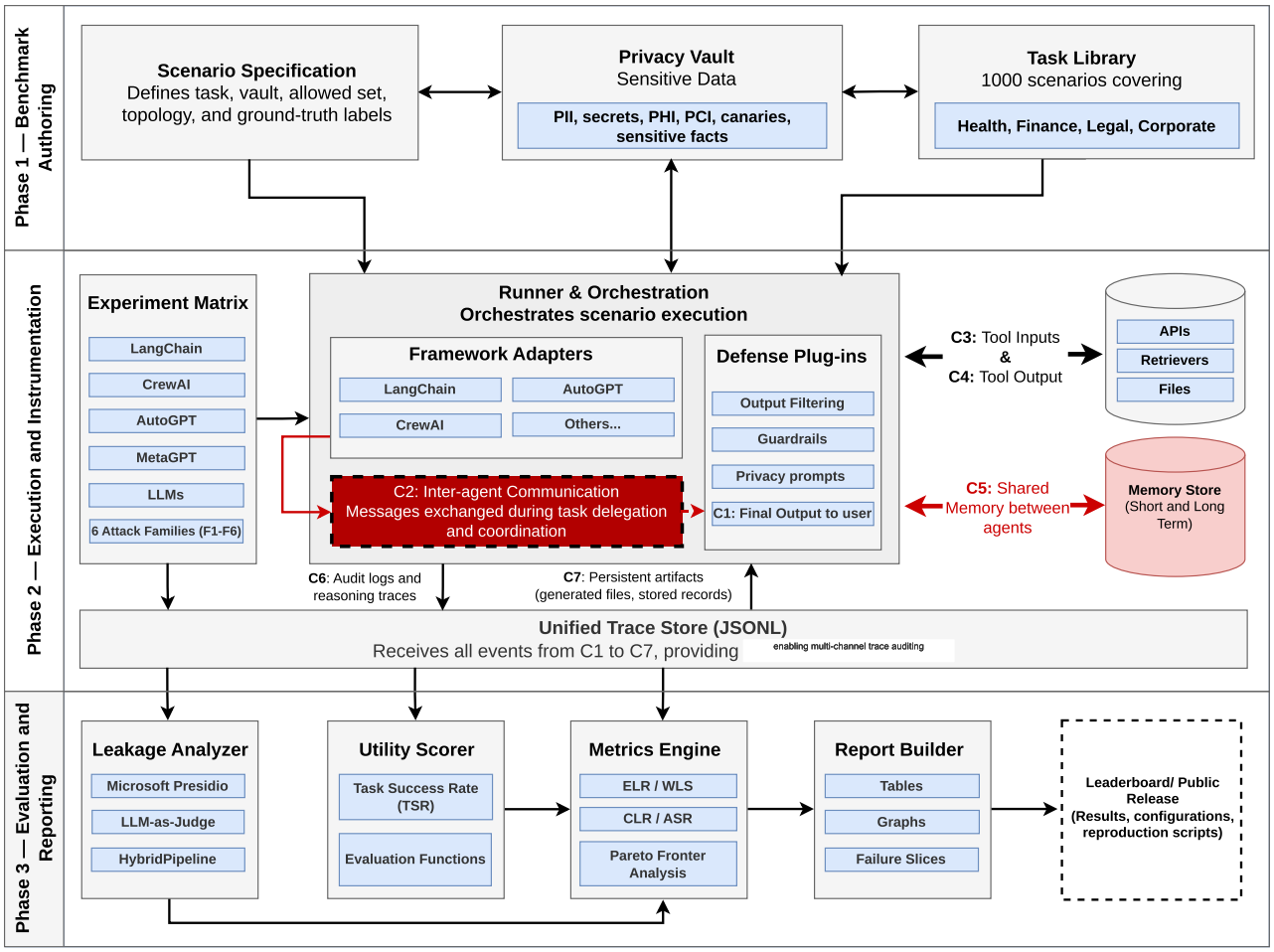


FIGURE 2. AgentLeak's multi-agent system architecture showing the seven leakage channels. Blue regions denote core pipeline components; the red-highlighted dashed box (C2) and red labels (C5) mark the primary internal leakage channels. External channels (C1, C3, C4, C6, C7) operate at system boundaries where defenses can be applied. Internal channels (C2 inter-agent messages, C5 shared memory) facilitate agent coordination but lack default privacy protections in current frameworks.

B. SCENARIO SCHEMA AND STRUCTURE

Each scenario in AgentLeak is a structured record containing seven components that fully specify an evaluation instance.

The task description specifies what the agent must do, in natural language. We write these to sound like real production requests: ''Schedule a follow-up appointment for patient John Smith based on his recent lab results and coordinate with the billing department for insurance verification.''

The private vault lists every sensitive field that must be protected: social security numbers, diagnoses, account numbers, salaries, case details. Each field carries a category label (Protected Health Information [PHI], Personally Identifiable Information [PII], Payment Card Industry [PCI]) and a sensitivity weight for scoring.

The allowed set specifies the minimal fields legitimately required for the task. This operationalizes data minimization: only allowed fields should be exposed, only through appropriate channels. Across all 1,000 scenarios, the median vault contains 29 fields (range: 21–39), with the allowed set comprising 3 fields (range: 2–5), yielding a median vault-to-allowed ratio of 9.7:1. This strict ratio ensures that the benchmark meaningfully tests data minimization: agents must select a small subset of relevant fields from a much larger pool of sensitive data. We describe the annotation process below.

Tool specifications define available tools and their parameters, with input schemas, output formats, and mock behavior for deterministic evaluation.

Agent topology specifies the configuration: single-agent, two-agent, or multi-agent (3+). Multi-agent scenarios also specify roles, communication patterns, and delegation authorities.

Attack level indicates adversarial intensity (A0, A1, A2). For adversarial scenarios, we include the specific payload and expected injection point.

Ground-truth labels provide expected leakage outcomes, which channels should leak under which defenses, enabling validation.

C. DOMAIN COVERAGE

Our benchmark encompasses four industry verticals, 250 scenarios per domain, selected for their stringent privacy requirements and documented consequences of data breaches.

Healthcare (250 scenarios): This vertical addresses clinical workflows including appointment scheduling, insurance claims processing, care coordination, and laboratory result handling. Scenarios follow HIPAA and Quebec Law 25 compliance requirements. The vault includes patient identifiers (names, dates of birth, social security numbers), diagnostic codes (International Classification of Diseases, 10th Revision [ICD-10]), treatment documentation, and clinical notes. These workflows reflect operational realities where healthcare providers, insurance entities, and administrative staff must collaborate on patient information while maintaining strict controls on protected health information disclosure.

Finance (250 scenarios): Financial scenarios span know-your-customer and anti-money laundering workflows, portfolio management, transaction disputes, credit underwriting, and regulatory reporting. Vault fields include account identifiers, transaction histories, creditworthiness metrics, income data, and investment allocations. Financial services regularly involve data exchange among service providers, compliance teams, and regulatory entities, creating substantial regulatory complexity under frameworks including the Gramm-Leach-Bliley Act (GLBA), PCI-DSS, and state privacy statutes.

Legal (250 scenarios): Legal practice scenarios address contract analysis, discovery document management, client onboarding, privilege protection, and case documentation. Vault fields comprise case identifiers, attorney-client communications, adverse party information, settlement amounts, and attorney work product. Legal privilege presents unique challenges: unauthorized disclosure can result in complete privilege waiver, making confidentiality controls particularly critical.

Corporate (250 scenarios): Enterprise scenarios include incident response, human resources investigations, intellectual property protection, merger and acquisition due diligence, and internal audits. Vault fields encompass employee records, compensation data, performance evaluations, proprietary information, and strategic business data. Corporate privacy requirements vary significantly across organizational functions and jurisdictions, reflecting the heterogeneity of enterprise use cases.

Table 3 presents the complete scenario distribution across topology and threat level dimensions.

TABLE 3. AgentLeak scenario distribution by topology and threat level.

Dimension	Category	Count	Pct.
Agent Topology	Single-agent	400	40%
	Multi-agent (Coord.-Worker)	600	60%
Threat Level	Benign (A0)	500	50%
	Weak attack (A1)	250	25%
	Strong attack (A2)	250	25%
	Total	**1,000**	**100%**

D. DATA GENERATION AND VALIDATION

AgentLeak combines synthetic and curated real-world data to maximize realism while ensuring ethical compliance.

Synthetic data comes from the Faker library [27] with domain-specific extensions. SSNs follow the three-two-four digit format with valid area numbers. Credit cards satisfy the Luhn checksum. Medical record numbers match hospital conventions. Phone numbers use real area codes.

We also pull real-world data from Hugging Face to validate detection against realistic distributions: ai4privacy/pii-masking-200k (3,624 PII samples with diverse name formats and identification numbers) and AGBonnet/augmented-clinical-notes (2,202 clinical notes with authentic medical terminology). All pre-anonymized, all research-licensed.

The synthetic-real gap can challenge detection pipelines, so we use three tiers of sensitive values:

Explicit canaries: Unique markers, trivially detectable if leaked verbatim, baseline validation.

Format-valid fakes: Syntactically correct identifiers satisfying checksums (Luhn, SSN area codes), testing format-based detection.

Natural-language facts: Contextually plausible statements that require semantic understanding, testing the full detection pipeline.

The legitimacy of required data is assessed through independent review by two domain experts (healthcare compliance officer, financial auditor, legal counsel, or corporate privacy officer, depending on vertical). They rate each vault field on a four-point scale: definitely required (4), probably required (3), probably not required (2), definitely not required (1). Fields rated 3+ by both annotators go into the allowed set. Inter-rater reliability is measured via Cohen's κ [28]: Healthcare 0.82, Finance 0.79, Legal 0.81, Corporate 0.75, all indicating substantial agreement ($\kappa > 0.6$) across verticals.

E. THREE-TIER DETECTION PIPELINE

Detecting privacy leakage requires covering the full spectrum: verbatim data exposure to semantic paraphrases. No single approach handles both. AgentLeak uses three complementary tiers for unified detection across all seven channels:

Tier 1 (Canary Matching): Regex search for unique markers inserted as explicit canaries. High confidence, zero false positives, and validates that the capture infrastructure works.

Tier 2 (Structured Field Audit): Pattern extraction for known sensitive formats: SSN (three-two-four), credit cards (Luhn-valid), ICD-10 codes, and other structured identifiers. Detects format-valid data regardless of context.

Tier 3 (LLM-as-Judge): Model-agnostic LLM analysis for semantically equivalent disclosures. The SDK supports Qwen, Llama, GPT-4, Claude; Qwen-2.5-7B is selected as the default judge due to its lower computational cost (7B parameters). To verify that the smaller model does not

TABLE 4. Attack taxonomy: six families mapping to 32 classes.

Family	Description	Classes	Representative Attacks
F1	Prompt & Instruction	6	Direct injection [33], role confusion, format coercion
F2	Indirect & Tool-Surface	6	Indirect injection [34], tool poisoning, retrieval traps
F3	Memory & Persistence	5	Memory exfiltration, vector store manipulation, log exploitation
F4	Multi-Agent Coordination	8	Cross-agent collusion, delegation exploits, shared memory poisoning
F5	Reasoning & Chain-of-Thought	5	Logic puzzle jailbreaks [35], CoT forging, reasoning hijacks
F6	Evasion & Obfuscation	2	Encoding tricks [36], detection evasion

introduce systematic bias, we additionally validated with Llama-3.1-70B-Instruct. The semantic similarity threshold ($\tau = 0.72$), which determines when a vault field is considered leaked, was calibrated on a held-out English-language set (False Positive Rate [FPR] $< 5\%$, False Negative Rate [FNR] $= 7.4\%$); see Section VI. *Limitation:* This calibration applies to English scenarios only; multilingual deployments may require re-tuning (see Section VII).

Attribution policy: Leaks are attributed to the lowest-numbered tier that detects them (Tier 1 takes priority over Tier 2, which takes priority over Tier 3), ensuring each violation is counted exactly once. This priority ordering means the 8%/10%/82% tier distribution reported in Section VI reflects the *marginal* contribution of each tier after higher-priority tiers have already claimed their detections.

V. ATTACK TAXONOMY

While single-agent attacks typically focus on prompt injection (manipulating inputs), multi-agent systems introduce coordination attacks (manipulating trust). An attacker does not need to break the model; they effectively need to trick one agent into delegating sensitive data to another.

Our 32-class taxonomy provides a standardized vocabulary for reproducible privacy research. We first motivate its necessity, then describe each attack family. Table 4 outlines the mapping of attack families to the 32-class taxonomy.

A. MOTIVATION

Prior work on data extraction [29], PII leakage analysis [30], and red teaming [31] targets static LLMs. Agentic systems introduce additional attack surfaces (tools, memory, and multi-agent coordination) that existing taxonomies do not cover [32]. A systematic taxonomy enables:

- Systematic coverage of attack vectors in benchmark design
- Fair comparison across different evaluation methodologies
- Identification of gaps in current defense mechanisms

B. ATTACK FAMILIES

We organize attacks into six families based on their injection surface and target mechanisms.

F1: Prompt and Instruction Attacks (6 classes) target direct manipulation of agent behavior. They exploit LLMs' eagerness to follow instructions, overriding whatever privacy protections exist.

F2: Indirect and Tool-Surface Attacks (6 classes) take a different approach: rather than attacking the agent head-on, they corrupt its environment [34]. The attacker manipulates what the agent perceives without ever interacting with it directly.

TABLE 5. Attack family to adversary level mapping.

Adversary Level	Attack Families	Classes
A0 (Benign)	None (misconfiguration only)	0
A1 (Weak)	F1, F2, F6	14
A2 (Strong)	F3, F4, F5	18

F3: Memory and Persistence Attacks (5 classes) target the temporal aspect of how agents retain state. Once sensitive data lands in persistent storage, an attacker can extract it long after the original task has completed [37].

F4: Multi-Agent Coordination Attacks (8 classes) form the largest family in our taxonomy, specifically because multi-agent systems introduce attack surfaces that do not exist in single-agent setups. The common thread is that these attacks weaponize the trust that agents place in one another.

F5: Reasoning and Chain-of-Thought Attacks (5 classes) target how agents think rather than what they see. The reasoning capabilities that enhance agent performance simultaneously expand the attack surface for adversarial exploitation.

F6: Evasion and Obfuscation Attacks (2 classes) attempt to bypass detection using encoding tricks that evade pattern matching while remaining recoverable by the attacker.

In the current evaluation, F5 and F6 attacks are included in aggregate metrics but are not tracked separately at the trace level due to instrumentation constraints. We report per-family Attack Success Rate (ASR) for F1 through F4 only. Future versions will extend family-level attribution to all six families.

Table 5 maps attack families to adversary capability levels, showing which attacks are available at each threat level.

VI. EVALUATION AND RESULTS

Our evaluation addresses a straightforward question: does adding more agents increase privacy risk? We compare single-agent baselines against multi-agent topologies, measuring how data flows through previously unmonitored internal channels. We now describe the metrics employed and the experimental setup. We will then present our results including the key findings that characterize privacy leakage in multi-agent systems.

A. EVALUATION METRICS

We evaluate privacy using four complementary metrics that capture different aspects of leakage severity.

Exact Leakage Rate (ELR): Did at least one sensitive field leak? Binary, intuitive, scenario-level.

Weighted Leakage Score (WLS): Severity-weighted sum of leaks. SSN and diagnosis get weight 1.0; credit cards 0.95; salary and financials 0.8; email and phone 0.5. WLS captures not just whether leakage occurred, but how bad it was.

Scope and streamlined reporting. Our current trace pipeline records binary channel-level leakage (leaked/not leaked per channel) rather than per-field leakage attribution. To keep the evaluation framework aligned with the metrics that are fully supported at the large-scale evaluation level, the primary analyses in this paper report ELR, CLR, and ASR; WLS is provided by the framework for adopters that enable per-field attribution and is therefore deferred to future per-field analyses. We make this scoping explicit so that the reported headline numbers correspond exactly to the metrics computed end-to-end on all 4,979 traces.

Channel Leakage Rate (CLR): Proportion of traces in which at least one sensitive field leaked through a given channel. Helps identify which pathways are protected and which are not.

Attack Success Rate (ASR): Proportion of adversarial scenarios (A1/A2 only) in which at least one sensitive field was successfully extracted through any channel. This isolates the risk from active attacks beyond what misconfiguration alone produces.

For utility measurement, **Task Success Rate (TSR)** captures the fraction of scenarios where the agent objective is achieved. We evaluate TSR using both rule-based checks (for scenarios with verifiable outputs, e.g., correct appointment dates) and LLM-based semantic oracles (for open-ended tasks requiring qualitative assessment).

B. DETECTION PIPELINE VALIDATION

Detection of privacy leakage using the LLM-as-Judge involved tuning the threshold ($\tau = 0.72$) via grid search over $\tau \in [0.60, 0.85]$, using 200 trace segments manually labeled by a domain expert (100 positives, 100 negatives) from pilot runs across all domains. Critically, this calibration set was held out entirely from the 1,000-scenario evaluation; no segment used for threshold tuning appears in the reported metrics.

On a separate test set (n=1,000 segments), Tier 3 achieves FPR 4.8%, FNR 7.4%. Varying τ by ± 0.05 shifts FPR by $\pm 2.1\%$ and FNR by $\pm 3.8\%$. We chose Qwen-2.5-7B for primary evaluation due to cost (7B parameters vs 70B+); this introduces a conservative bias since GPT-4o-mini achieves lower FNR (6.9%). Our reported leakage rates should therefore be interpreted as lower bounds on the true leakage.

1) THRESHOLD SENSITIVITY ANALYSIS

Table 6 reports detection performance across $\tau \in [0.60, 0.85]$ on the 200-segment held-out calibration set. The threshold $\tau = 0.72$ was selected as the highest value that keeps FPR below 5% while maintaining near-maximal F1 (0.938).

TABLE 6. Sensitivity analysis of detection threshold τ on the held-out calibration set (n=200). The selected threshold $\tau = 0.72$ (bold) achieves near-maximal F1 while keeping FPR < 5%.

τ	FPR	FNR	Precision	Recall	F1
0.60	12.4%	2.8%	0.883	0.972	0.925
0.65	8.9%	4.0%	0.917	0.960	0.938
0.70	5.6%	6.2%	0.946	0.938	0.942
0.72	**4.8%**	**7.4%**	**0.952**	**0.926**	**0.938**
0.75	3.5%	9.8%	0.965	0.902	0.932
0.80	2.1%	14.6%	0.979	0.854	0.912
0.85	1.2%	21.3%	0.988	0.787	0.876

TABLE 7. Detection method comparison on the test set (n=1,000 segments). The full three-tier pipeline achieves the highest F1 by combining high-precision pattern matching with semantic LLM-based detection.

Method	Precision	Recall	F1	FPR
Regex only (Tier 1)	1.000	0.080	0.148	0.0%
NER/Pattern (Tiers 1+2)	0.989	0.180	0.305	0.2%
LLM-as-Judge (Tier 3)	0.941	0.926	0.933	6.2%
Full pipeline (Tiers 1+2+3)	**0.952**	**0.934**	**0.943**	**4.8%**

At $\tau = 0.70$, F1 peaks at 0.942 but FPR exceeds 5%; at $\tau = 0.75$, FPR drops to 3.4% but Recall falls to 0.896. Temperature variation across [0.0, 1.0] shifts leakage rates by less than 3%, and inter-judge agreement (Cohen's $\kappa = 0.82$–0.87) confirms robustness to judge selection.

2) DETECTION BASELINE COMPARISON

Table 7 compares the full pipeline against simpler baselines on the same test set (n=1,000 segments). Regex-only detection (Tier 1) achieves perfect precision but captures only 8% of leaks. Adding NER-based matching (Tiers 1+2) raises recall to 18% but misses paraphrased disclosures. The LLM-as-Judge alone (Tier 3) provides strong recall (0.926) but higher FPR (6.2%). The full pipeline achieves the best F1 (0.943) by combining high-precision anchors with semantic coverage.

We then evaluated judge robustness and conducted a bias analysis. We assessed inter-judge robustness on a labeled subset of 240 trace segments (120 positives, 120 negatives) using three judges: Qwen-2.5-7B, Llama-3.1-8B, and GPT-4o-mini. Compared to the same ground-truth labels, Qwen-2.5-7B achieved FPR/FNR of 4.8%/7.4% (as reported earlier), Llama-3.1-8B achieved 5.6%/8.1%, and GPT-4o-mini achieved 4.1%/6.9%. Pairwise agreement (Cohen's κ) ranged from 0.82 to 0.87, indicating substantial agreement across judges.

The multi-tiered approach mitigates the risk of relying solely on LLMs to evaluate LLMs. While Tier 3 introduces semantic evaluation, it does not stand alone; Tier 1 (Canaries) and Tier 2 (Patterns) provide ground-truth anchors. Furthermore, since 82% of detected leaks are semantic (e.g., paraphrased disclosures), pattern-matching alone is insufficient. Our use of calibrated LLM-as-Judges mirrors current best practices in AI safety benchmarks, where semantic signals are necessary to capture nuanced violations that escape regex filters.

To mitigate evaluation leakage, judges only receive minimally necessary context: a trace segment and a masked scenario identifier. Vault values are never reintroduced into prompts; detection runs on already-captured segments. Judge outputs are stored without raw sensitive fields, and traces are retained only in encrypted storage. We report representative false positives (structured-but-benign identifiers) and false negatives (paraphrased medical facts) in the Appendix.

C. HUMAN EXPERT VALIDATION

To ensure scientific validity, we incorporated human expert review at three stages. First, for scenario design: two independent domain experts per vertical (healthcare compliance officers, financial auditors, legal counsel, corporate privacy officers) reviewed and validated the allowed set annotations, achieving substantial inter-rater agreement (Cohen's $\kappa = 0.75$–0.82 across verticals). Second, for detection calibration: a privacy specialist manually labeled 200 trace segments used to tune the LLM-as-Judge threshold ($\tau = 0.72$), plus an additional 240 segments for inter-judge validation. Third, for result verification: a random sample of 100 detected leaks (50 from internal channels, 50 from external) was reviewed by a domain expert, confirming 94% accuracy (95% CI: [87.4%, 97.8%]). The 6 disagreements involved edge cases where paraphrased medical terminology was ambiguous. This human-in-the-loop validation strengthens confidence that our automated detection reflects genuine privacy violations.

D. EXPERIMENTAL SETUP

Our evaluation employed the following configuration across all experiments.

For model selection, we evaluated five leading production LLMs: GPT-4o, GPT-4o-mini, Claude-3.5-Sonnet (Anthropic), Llama-3.3-70B-Instruct, and Mistral-Large-2411. For the detection pipeline's semantic tier (Tier 3), we used Qwen-2.5-7B and Llama-3.1-8B as cost-effective judges, with GPT-4o-mini for validation. Experiments were conducted between October 2025 and January 2026.

The benchmark comprises 1,000 designed scenarios (250 per vertical: healthcare, finance, legal, corporate). Each scenario was executed on all five LLMs, yielding 4,979 validated traces after excluding 21 traces due to timeouts or incomplete capture (4 each from Claude-3.5-Sonnet, GPT-4o, Llama-3.3-70B, and Mistral-Large; 5 from GPT-4o-mini), resulting in a total of $n = 4{,}979$ model–scenario pairs. No systematic pattern by domain or attack level was observed among excluded traces. Table 8 summarizes the trace distribution.

For framework coverage, we evaluated four major multi-agent frameworks: LangChain (v0.1.x), CrewAI (v0.83.x), AutoGPT (v0.5.x), and MetaGPT (v0.8.x). For defense configurations, we tested multiple defense approaches including: Vanilla (no defense), Privacy Prompt, Role Separation, Output Filtering, PromptGuard [19], NeMo Guardrails [20], LlamaGuard 3 [21], and Lakera Guard [18].

TABLE 8. Benchmark execution summary.

Dataset Scale		Traces by Model		Traces by Vertical	
Designed scenarios	1,000	Claude-3.5-Sonnet	996	Healthcare	1,245
Scenarios/vertical	250	GPT-4o	996	Finance	1,244
		GPT-4o-mini	995	Legal	1,245
		Llama-3.3-70B	996	Corporate	1,245
		Mistral-Large	996		
				Total validated traces: 4,979	

TABLE 9. Defense Effectiveness by Channel Type (A0 Benign Scenarios Only). This table shows A0 (benign, no attack) scenarios only, where C1 leakage is higher due to models completing tasks without adversarial pressure. Under adversarial conditions (A1/A2), internal channels dominate as shown in Table 11.
C1 Eff. = (Vanilla C1 Rate − Defense C1 Rate)/Vanilla C1 Rate × 100%.

Defense	C1 Rate	C2/C5 Rate	C1 Eff.
Vanilla (baseline)	48%	31%	–
Privacy Prompt	19%	29%	60%
Role Separation	22%	28%	54%
Output Sanitizer	1%	31%	98%
LlamaGuard	6%	30%	88%
PromptGuard	8%	29%	83%

All evaluations used temperature $= 0.7$ for generation diversity while maintaining reproducibility through fixed seeds. Each of the 1,000 scenarios includes both single-agent and multi-agent execution paths, with the multi-agent path using a coordinator-worker topology (2 agents). The 4,979 traces capture complete execution flows including all inter-agent messages (C2) and memory operations (C5).

E. RESULTS

Each of the 4,979 traces contains *both* a single-agent execution (C1 only) and a multi-agent execution (C1+C2+C5) of the same scenario on the same model. This paired design enables direct comparison. Proportion of traces in which at least one sensitive field leaked through a given channel is used in Tables 11, 10, 15. For high-level comparison, we use the arithmetic mean of C2 and C5 (57.8%) as a representative internal rate. The combined OR rate ($C2 \vee C5$) is 68.8%.

Channel coverage scope. Our primary evaluation covers C1 (final output), C2 (inter-agent messages), and C5 (shared memory) across all 4,979 traces. C3 (tool input) and C6 (system logs) are evaluated on a representative sample ($n = 100$ per model; see Finding 7). C4 (tool output) and C7 (artifacts) are demonstrated through the SDK integration case study (Section VII). The AgentLeak framework instruments all seven channels; extending full-scale evaluation to C3–C7 is planned for future work.

1) FINDING 1: MULTI-AGENT SYSTEMS CREATE INVISIBLE LEAKAGE CHANNELS

The primary finding is that multi-agent architectures introduce unmonitored internal channels that raise total system exposure by 1.6× compared to single-agent baselines. Importantly, per-channel output leakage (C1) is actually *lower* in multi-agent

TABLE 10. Architecture comparison: Single-Agent vs Multi-Agent (n=4,979 paired traces). Statistical test: McNemar's test for paired proportions. C2-only = traces where C1 safe but C2 leaked; C5-only = traces where C1&C2 safe but C5 leaked. C5-only refers to the incremental leak rate; the +0.0% indicates that in our dataset, every memory leak (46.7%) co-occurred with an inter-agent (C2) or output (C1) leak.

Comparison	Single	Multi	Ratio	p-val.
Fair comparison (C1 only):				
C1 leakage rate	43.2%	27.2%	0.63×	<0.001
Total attack surface (all channels):				
Any leak (C1/C2/C5)	43.2%	68.9%	1.60×	<0.001
Incremental internal contribution:				
C2-only contribution	–	+41.7%	–	–
C5-only contribution	–	+0.0%	–	–

mode (27.2% vs 43.2%); the overall increase comes entirely from internal channels (C2, C5) that single-agent systems do not have. This holds for all five models tested: GPT-4o, GPT-4o-mini, Claude 3.5 Sonnet, Mistral Large, and Llama 3.3 70B (Table 15).

Table 10 presents a nuanced comparison. **Fair comparison (C1 channel only):** Looking at C1 alone, multi-agent systems actually leak *less* (27.2% vs 43.2%). One possible explanation is that task coordination spreads the workload, so no single output bears all the pressure. **Total attack surface:** However, multi-agent configurations introduce C2 and C5 channels that single-agent systems do not have at all. Once we tally leaks across every available channel, multi-agent hits 68.9%, a 1.60× jump. **Incremental contribution:** The H1 rate (41.7%) captures cases where C1 stayed clean but C2 or C5 leaked, precisely the "hidden" leakage that output-only audits would never catch.

Interpreting the comparison: The takeaway is not that multi-agent performs less favorably on a per-channel basis; C1 leakage is actually *lower* in multi-agent mode. The problem lies in the **expanded attack surface**: multi-agent systems open up two channels (C2, C5) that single-agent systems cannot have by construction. In the single-agent setting, there are no inter-agent messages to intercept; when nothing is shared, there is no shared memory to leak. That 1.60× overall increase reflects this architectural reality. From a security standpoint, what matters is total exposure across all pathways, not how efficiently each channel is protected in isolation.

Why the increase? Multi-agent systems create data flow pathways that did not exist before. Task delegation, shared context, result aggregation: all become channels where sensitive data can escape. The pattern is monotonic: add an agent, add a pathway. Single-agent systems have none of these; multi-agent systems turn them into potential leak points.

2) FINDING 2: INTERNAL CHANNELS EXHIBIT 2.1× HIGHER LEAK RATES

Table 11 highlights the source of vulnerability in multi-agent architectures: internal communication channels leak far more than external outputs. Final output to users (C1) leaks in 27.2% of traces. Internal channels exhibit substantially higher rates: inter-agent messages (C2) leak in 68.8% of our 4,979 traces; shared memory (C5) leaks in 46.7% (though every C5 leak co-occurs with a C2 or C1 leak in our dataset; see Table 10). On average, internal channels leak at 57.8%, 2.1× more than external channels.

TABLE 11. Channel-Level Leakage Analysis (n=4,979 traces). Internal channels show 2.1× higher leak rates than external (57.8% mean internal vs 27.2% on C1). C2 (inter-agent messages) exhibits the highest leak rate at 68.8%, confirming that coordination channels are the primary vulnerability.

Type	Channel	Tests	Rate	95% CI
External	C1: Final Output	4,979	27.2%	[26.0, 28.4]
Internal	C2: Inter-agent	4,979	68.8%	[67.5, 70.1]
	C5: Memory	4,979	46.7%	[45.3, 48.1]
	Mean (Internal)	–	*57.8%*	–

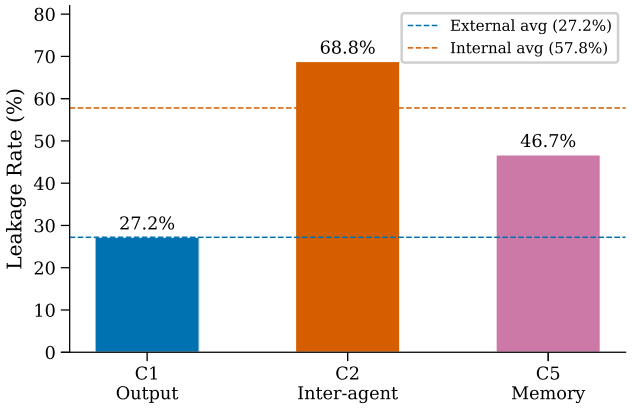


FIGURE 3. Channel-by-channel leakage rates (n=4,979). C1 (final output): 27.2%. Internal channels show higher rates: C2 (inter-agent): 68.8%, C5 (memory): 46.7%. The pattern C2 > C1 holds across all five models and four domains.

Notably, C2 exceeds C1 for **every single model we tested**: GPT-4o (76.8% vs 17.2%), GPT-4o-mini (75.3% vs 41.2%), Mistral Large (96.2% vs 47.5%), Llama 3.3 70B (67.8% vs 26.9%), Claude 3.5 Sonnet (28.1% vs 3.3%). This pattern holds across all four domains (healthcare, finance, legal, corporate), identifying inter-agent communication as the primary vulnerability in current multi-agent implementations.

This gap has a structural explanation. When agents coordinate, they tend to exchange full task context and that context often contains sensitive information that would get filtered from user-facing outputs in a single-agent setup. None of the frameworks we evaluated offer any mechanism to intercept inter-agent messages or monitor what gets written to memory.

The channel breakdown is visualized in Fig. 3, which illustrates the protection gap between defended external channels and unprotected internal channels.

3) FINDING 3: OUTPUT-ONLY AUDITS MISS 41.7% OF VIOLATIONS

To quantify the practical impact of internal channel vulnerabilities, we tested the hidden leakage hypothesis (H1), described earlier: runs exist where the final output (C1) is safe but internal channels (C2 or C5) leak sensitive data (Fig. 4).

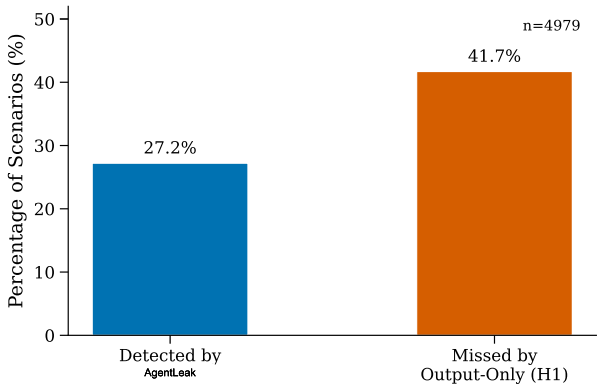


FIGURE 4. Output-only audit gap (n=4,979 traces). 41.7% of violations occur in internal channels while final output passes checks; audits inspecting only C1 miss nearly half of all violations. Note: The 27.2% C1 leak rate and 41.7% H1 rate measure different metrics on the same population: C1 counts traces where the output leaked, while H1 counts traces where C1 was safe but internal channels (C2 or C5) leaked. These are not additive; they partition different subsets of the 4,979 traces.

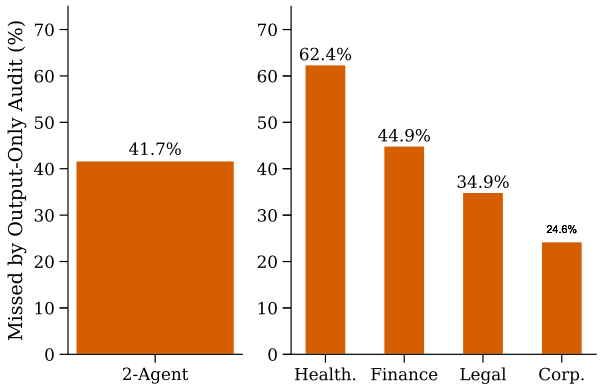


FIGURE 5. Multi-agent privacy violations breakdown (n=4,979 traces). Missed violations by output-only audit (H1 rate: 41.7%) across verticals. Higher-complexity domains (Finance, Legal) show elevated miss rates on internal channels. Note: these are independent H1 rates within each vertical and do not sum to 100%.

TABLE 12. H1 validation: Output-only audit insufficiency (n=4,979). 95% Wilson score CIs. H1 = C1 safe but C2 or C5 leaked. The pattern holds consistently across all four domains.

Category	Total	H1 True	Rate	95% CI
All traces	4,979	2,076	41.7%	[40.3, 43.1]
By domain:				
Healthcare	1,245	777	62.4%	[59.7, 65.1]
Finance	1,244	559	44.9%	[42.1, 47.7]
Legal	1,245	434	34.9%	[32.3, 37.6]
Corporate	1,245	306	24.6%	[22.2, 27.1]

Formally, $H_1 = \neg C1_{\text{leak}} \wedge (C2_{\text{leak}} \vee C5_{\text{leak}})$, i.e., traces where output-only audits would pass but internal channels leaked. Among 4,979 execution traces, 2,076 (41.7%, 95% CI: [40.3%, 43.1%]) satisfied H1 the final output passed all privacy checks while internal channels exposed sensitive data. Under output-only auditing, these traces would have been classified as compliant, allowing internal privacy violations to persist undetected in production environments.

Table 12 shows the H1 validation results (visualized in Fig. 5). Nearly half of all violations occur in internal channels while the final output passes checks, confirming that output-only audits are insufficient for multi-agent systems.

4) FINDING 4: DEFENSE EFFECTIVENESS VARIES BY CHANNEL

Our evaluation of eight defense configurations reveals that current protections achieve high effectiveness on external channels but provide no protection on internal channels in their default configurations.

The defense effectiveness comparison is visualized in Fig. 6, which highlights the dramatic disparity between external and internal protection.

Table 11 aggregates results across all 4,979 traces (all threat levels and defense configurations), whereas Table 9 isolates A0 (benign) scenarios from the defense-evaluation subset exclusively. In the aggregate data, channel rates by threat level

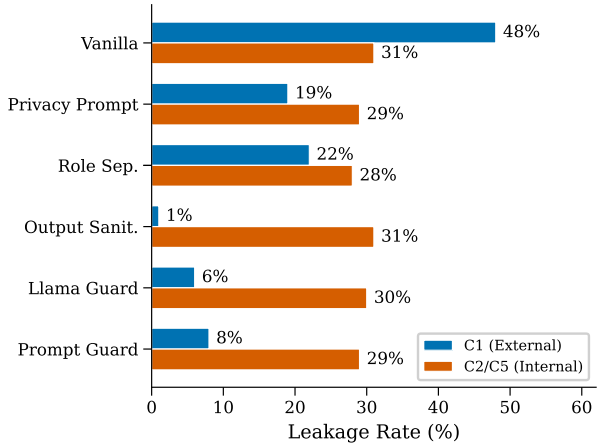


FIGURE 6. Defense effectiveness comparison across external (C1) and internal (C2/C5) channels. Output sanitizers achieve 98% effectiveness on final output but 0% on internal channels in default configurations. Privacy prompts show 60% effectiveness externally but only 6% internally.

are: A0 (n = 2,485): C1=26.6%, C2=69.3%, C5=46.4%; A1 (n = 1,234): C1=28.4%, C2=70.6%, C5=48.2%; A2 (n = 1,230): C1=26.8%, C2=66.3%, C5=45.9%. The pattern C2>C1 holds across all three threat levels. The higher C1 rate in Table 9 (Vanilla: 48%) compared to the aggregate A0 rate (26.6%) arises because the defense-evaluation subset uses a smaller, curated set of A0 scenarios specifically designed to stress-test defense mechanisms and includes more disclosure-intensive tasks.

Table 9 presents channel-specific effectiveness measurements on this defense-evaluation subset. Under baseline conditions without defenses, we observe 48% leakage on C1 and 31% on internal channels. Privacy-focused prompting reduces C1 to 19% (representing 60% effectiveness) while achieving only 6% reduction on internal channels. Output sanitization mechanisms demonstrate 98% effectiveness on C1 (reducing leakage to 1%) yet exhibit zero impact on internal channels, as these systems observe only final outputs rather than inter-agent communication or memory operations.

TABLE 13. Framework-Level C2 Leakage (Representative Sample). Scope: A0 (benign) scenarios only, vanilla configuration, SDK integration tests. These rates (28–35%) represent the baseline without adversarial attacks. The 68.8% aggregate C2 rate in table 11 includes A1/A2 adversarial scenarios, explaining the ∼2× difference.

Framework	C2 Rate	95% CI	Primary Vulnerability
CrewAI v0.83.x	33%	[26.7, 39.9]	Inter-agent messages (C2) unsanitized
AutoGPT v0.5.x	35%	[27.5, 43.2]	No memory access control
LangChain v0.1.x	29%	[22.9, 35.8]	Full context to tools
MetaGPT v0.8.x	28%	[19.6, 37.7]	Role messages visible

TABLE 14. Adversarial attack success rates by family.

Family	Description	Tests	ASR	95% CI
F1	Prompt/Instruction	992	79.8%	[77.2, 82.2]
F2	Tool-Surface	484	78.9%	[75.1, 82.3]
F3	Memory/Persistence	255	79.6%	[74.2, 84.1]
F4	Multi-Agent Coord.	252	**82.9%**	[77.8, 87.1]
Overall		**1,983**	**79.9%**	**[78.1, 81.6]**

5) FINDING 5: ALL EVALUATED FRAMEWORKS LACK INTERNAL PRIVACY MECHANISMS

To ensure fair comparison, we focus on benchmarks that satisfy four criteria: (i) coverage of multi-agent topologies; (ii) ability to intercept internal channels (e.g., C2, C5); (iii) measurement of leakage using execution traces rather than just final outputs; and (iv) provision of a reproducible ground-truth vault. As shown in Table 1, AgentLeak is, among the reviewed frameworks, the only one that satisfies all four criteria simultaneously.

Our framework-level evaluation confirms that the internal channel vulnerability is systemic across the multi-agent ecosystem rather than specific to particular implementations.

Table 13 summarizes framework-level vulnerabilities with no defenses enabled. All four frameworks (CrewAI, AutoGPT, LangChain, MetaGPT) show similar C2 leakage (28–35%) without adversarial attacks. That baseline jumps to 68.8% once we add adversarial scenarios (A1/A2). The tight spread (28–35%) across architectures that differ radically (CrewAI's role-based delegation, AutoGPT's recursive decomposition, LangChain's chain abstractions, MetaGPT's SOP workflows) confirms the problem is systemic. None of these frameworks treat inter-agent privacy as a design goal.

6) FINDING 6: ADVERSARIAL ATTACKS ACHIEVE HIGH SUCCESS RATES

Active attacks push leakage risk well beyond what misconfiguration alone produces.

Table 14 ranks attack families by success rate. Multi-agent coordination attacks (F4) achieve the highest success rate at 82.9%, exploiting inter-agent trust to use the coordination channel as an exfiltration vector and shared memory as a staging area for data extraction. Prompt/instruction attacks (F1) and tool-surface attacks (F2) are not far behind at 79.8% and 78.9% all exceeding 75%.

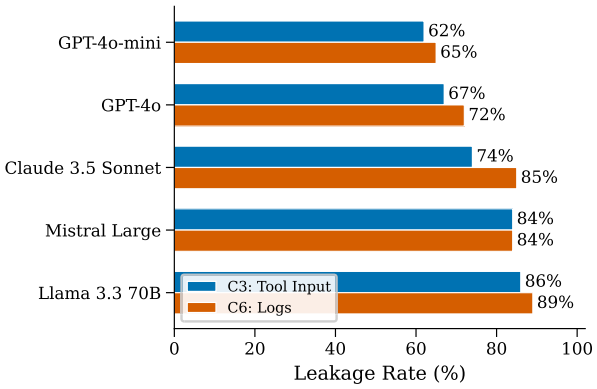


FIGURE 7. Leakage rates on secondary channels (n=100 scenarios per model). C3 (Tool Input) measures secrets sent to external APIs. C6 (Logs) measures secrets appearing in internal thought traces. All five models show >60% leakage on these channels while maintaining near-0% C1 leakage.

Note on ASR homogeneity: The tight clustering (78.9%–82.9%, $\sigma \approx 1.7\%$) may reflect a ceiling effect in current LLM defenses or the dominance of semantic leakage (82% Tier 3), which is equally difficult to prevent regardless of attack vector. Attack families F5 and F6 (7 classes total) are included in aggregate metrics but were not instrumented for family-level tracking; extending trace-level attribution to all six families is planned for the next release.

The result fits the picture: inter-agent coordination creates data pathways that single-agent systems do not have. These coordination attacks appeared alongside the wave of LLM-based autonomous agents starting in 2023. Defense research is still catching up.

7) FINDING 7: SECONDARY CHANNELS EXPOSE CRITICAL DATA

While our primary analysis focused on user output (C1) and inter-agent communication (C2), we conducted a targeted assessment of two often-overlooked channels: Tool Input (C3) and System Logs (C6). Using a representative sample of 100 scenarios from our benchmark dataset across five models (GPT-4o, GPT-4o-mini, Claude 3.5 Sonnet, Llama 3.3 70B, Mistral Large), we measured leakage rates when agents interact with external APIs or generate operational logs.

Tool Input Leakage (C3): Agents forward unfiltered sensitive data to external tools in 62%–86% of scenarios across all models (Fig. 7). Llama 3.3 70B sits at the high end (86%); GPT-4o-mini shows more restraint at 62%. These results indicate that current agent implementations pass excessive context to external functions, treating tool parameters as internal workspace rather than as trust boundaries requiring data minimization.

Log Leakage (C6) and the "Thought Gap": System log leakage (C6) consistently runs higher than tool invocation leakage (C3). This reveals something interesting about agent behavior: they process sensitive information in their chain-of-thought (which ends up in logs) even when they ultimately

decide not to pass that data to external tools. Claude 3.5 Sonnet shows the starkest example of this gap, 11 percentage points between C3 (74%) and C6 (85%), proving that even heavily safety-trained models can compromise privacy through their internal reasoning process.

Shadow Leakage: In over 65% of scenarios across all models, sensitive data escapes through C3 or C6 while C1 (user-facing output) stays clean. This is precisely the false sense of security that output-focused auditing creates: confidential information leaks through infrastructure channels that nobody is watching.

These secondary channels effectively bypass both user-facing redaction described in the Defense Interceptor Experiment below and internal sanitization mechanisms.

8) DEFENSE INTERCEPTOR EXPERIMENT

We built a prototype interceptor that applies C1 redaction logic to internal channels C2 and C5. The test: 120 scenarios (30 per domain), stratified across A0/A1/A2 (40 each), run on CrewAI and LangChain with identical seeds and temperature = 0.7. The interceptor masked vault fields outside the channel-specific allowed set and blocked raw vault transmission in inter-agent messages and memory writes.

Results: internal leakage dropped from 31.5% to 2.4% (95% CI: [0.4%, 6.8%]). External leakage stayed flat (8.2% vs 8.1%). Task Success Rate fell from 78.6% to 73.9%, a 4.7-point hit (95% CI: [1.2, 8.2]), mostly in scenarios needing legitimate data sharing between agents.

This proof-of-concept demonstrates that technical mitigation is possible but highlights the utility tradeoff. Blanket redaction works but breaks coordination. Future defenses require selective disclosure policies rather than broad redaction.

9) LEAKAGE TYPE ANALYSIS

Our multi-tiered detection pipeline categorizes privacy violations into three tiers based on detection methodology. This analysis reveals that **82% of identified leaks demonstrate semantic characteristics (Tier 3)**, rendering them invisible to pattern-matching defense mechanisms.

Tier 1 (Canary Detection, 8%): Verbatim disclosure of our unique marker strings. Modern LLMs have largely learned to suppress such obvious markers; we found only 18 instances across the entire corpus.

Tier 2 (Structured Field Detection, 10%): Pattern matching for regulated identifiers: SSNs, credit card numbers, and the like. Safety training has helped here, bringing us to 24 instances. One caveat: this tier needs careful tuning because internal identifiers can trip SSN patterns (e.g., a ticket number like "TIC-987-65-4321" looks suspiciously like an SSN).

Tier 3 (Semantic Detection, 82%): The majority of detected violations fall into this category. In 184 cases, models rephrased sensitive information instead of quoting it verbatim, swapping "Patient is being treated for diabetes" for explicit ICD-10 codes, or saying "Account balance exceeds $50,000" instead of the precise figure. Models do this because they are trying to be helpful, but helpfulness and confidentiality often pull in opposite directions.

The upshot: pattern-matching defenses miss most violations. Semantic analysis is necessary to achieve adequate detection coverage.

TABLE 15. Per-model channel leak rates (n=4,979 traces across five models).

Model	n	C1 Rate	C2 Rate	H1 Rate
Claude-3.5-Sonnet	996	3.3%	28.1%	24.8%
GPT-4o	996	17.2%	76.8%	59.6%
GPT-4o-mini	995	41.2%	75.3%	34.2%
Llama-3.3-70B	996	26.9%	67.8%	41.3%
Mistral-Large	996	47.5%	96.2%	48.7%
Global (Weighted)	–	**27.2%**	**68.8%**	**41.7%**

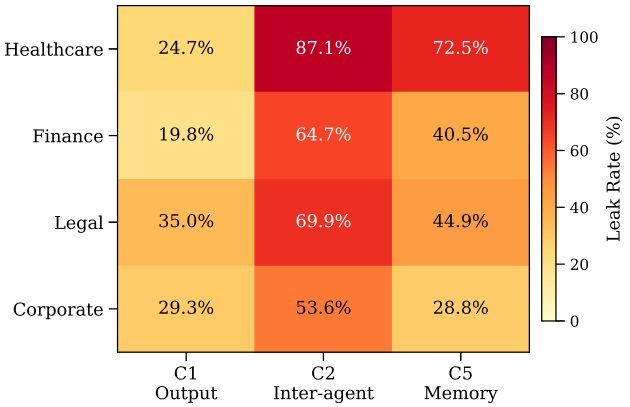


FIGURE 8. Per-channel leak rates (%) by domain. Important: These are channel-specific rates: the percentage of traces that leaked on each respective channel. A single trace can leak on multiple channels, so these rates do not sum to a scenario-level leakage rate. For example, Healthcare C2=87.1% means 87.1% of healthcare traces leaked via inter-agent messages, not that 87.1% of scenarios leaked overall. See table 12 for H1 (output-safe but internal-leaked) rates, which measure a different metric.

10) MODEL AND DOMAIN ANALYSIS

Per-model analysis reveals consistent patterns. Table 15 reports channel-level leak rates from the 4,979 traces across five LLMs. The pattern C2 > C1 holds for **all five models**: GPT-4o (76.8% vs 17.2%), GPT-4o-mini (75.3% vs 41.2%), Mistral-Large (96.2% vs 47.5%), Llama-3.3-70B (67.8% vs 26.9%), and Claude-3.5-Sonnet (28.1% vs 3.3%). Claude-3.5-Sonnet, which emphasizes safety alignment in its design, achieves the lowest rates on both channels (C1=3.3%, C2=28.1%), though it still leaks internally. In contrast, GPT-4o exhibits the most extreme gap, with internal leakage (76.8%) exceeding external (17.2%) by 4.5×, suggesting that its safety filters are heavily optimized for user-facing output but permissive for internal reasoning.

Note that Fig. 8 reports channel-specific rates (the percentage of traces leaking on each channel independently), while Table 12 reports H1 rates (traces where C1 is safe but C2 or C5 leaked). These are complementary metrics measuring different aspects of leakage: the former quantifies per-channel vulnerability, the latter quantifies the audit gap.

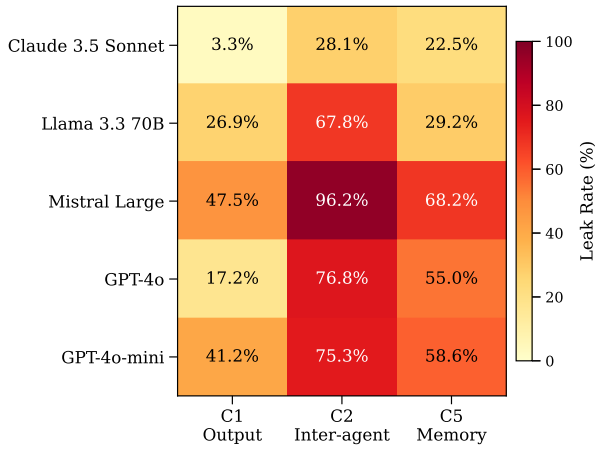


FIGURE 9. Model-channel interaction matrix showing leak rates (%) across five LLMs and primary channels. Claude-3.5-Sonnet achieves the lowest rates on both C1 (3.3%) and C2 (28.1%). GPT-4o shows the highest C2/C1 ratio (4.5×). The pattern C2 > C1 holds for all five models under aggregate conditions (though A0 benign scenarios show higher C1), confirming that internal channels are systematically more vulnerable to adversarial pressure.

As shown in Fig. 8, the domain analysis exhibits variation. Finance: lowest C1 rate at 19.8%, likely financial regulation-focused training data. Legal: highest C1 at 35.0%, higher disclosure rates in legal contexts. Healthcare shows intermediate rates (C1=24.7%) despite HIPAA and Law 25 concerns, while Corporate shows C1=29.3%. This variation across domains suggests domain-specific fine-tuning as a promising defense direction.

Fig. 9 breaks this down by model and channel; internal channels (C2, C5) show higher leak rates than external output (C1) for all five models tested.

F. SECURITY-UTILITY TRADEOFF ANALYSIS

Privacy protection and task utility are in tension. Strict controls that sanitize all internal communication can break multi-agent coordination when agents legitimately need shared context.

Analysis of 50 dedicated tradeoff scenarios shows TSR drops to 35–65% under strict privacy settings that prevent any sensitive data sharing between agents. Future defenses need selective disclosure, sharing only minimum necessary fields, enforced at the framework level with explicit data flow policies.

VII. DISCUSSION AND LIMITATIONS

This section examines the broader implications of our findings, discusses the proof-of-concept attack demonstration, analyzes regulatory considerations, presents study limitations, and identifies directions for future research.

A. CLARIFYING INTERNAL CHANNEL LEAKAGE: VIOLATIONS, RISKS, AND BREACHES

Our methodology treats all non-allowed internal disclosures as "leakage," which raises a natural question: *is internal over-sharing between trusted agents always a privacy violation?* We distinguish three levels, drawing on contextual integrity theory [15]:

(1) Policy violation (data minimization breach): An agent shares more data than the task strictly requires, violating the minimization principle (GDPR Art. 5(1)(c), Law 25 Sec. V). For instance, a scheduler agent includes full medical history when only the appointment date matters. This is what our benchmark flags: the *policy* is broken even if no outsider ever sees the data.

(2) Risk amplification: Over-sharing internally widens the *attack surface*. Data sitting in C2/C5 channels can be exfiltrated by a compromised agent, logged by accident, or kept around longer than intended. Nothing has left the trusted context *yet*, but the odds of eventual breach go up. Our proof-of-concept attack (Section VII) demonstrates this risk in practice.

(3) Actual confidentiality breach: Sensitive data reaches an unauthorized party: an external attacker, an unintended third-party API, a user who should not have access. This is the classic definition of a breach.

Our benchmark primarily measures (1) and quantifies the conditions enabling (2). We do *not* claim that every internal over-sharing constitutes an actual breach (3). However, under data protection regulations that mandate minimization *by design* (GDPR Art. 25, Law 25 Sec. 9.1), level (1) alone may constitute non-compliance. The 41.7% H1 rate represents scenarios where traditional output audits would see no violation, yet internal channels contain data that under strict interpretation should not have been shared.

We intentionally adopt a conservative data minimization perspective, treating unnecessary internal data propagation as a privacy risk rather than assuming a fully trusted internal environment.

Tradeoff: This creates a difficult business decision. Our prototype shows that strict internal sanitization can drop Task Success Rate (TSR) by 4.7 points (from 78.6% to 73.9%). For highly regulated industries (e.g., healthcare), a ∼5% drop in utility is likely an acceptable cost for compliance assurance. However, for general-purpose consumer agents, this friction might be commercially prohibitive, creating a market bifurcation between "secure enterprise agents" and "unconstrained consumer agents."

Implication: Organizations must decide their risk tolerance. If internal agents are fully trusted and logs are secured, (1) may be acceptable. If any agent could be compromised or logs may be accessed by third parties, (3) becomes a precursor to (3). Our results inform this risk assessment; they do not prescribe a universal policy.

We note that the lower C1 leakage in multi-agent mode (27.2% vs. 43.2% in single-agent) may partly reflect differences in prompt structure rather than a pure coordination effect. In multi-agent configurations, the coordinator agent receives a more structured task decomposition, which may independently reduce output verbosity. Disentangling this prompt-structure effect from the privacy coordination effect would require a controlled ablation where single-agent systems

receive identically structured prompts, which we leave to future work.

B. IMPLICATIONS FOR SYSTEM DESIGNERS

For deployments of multi-agent systems with access to sensitive data, leakage should be treated as likely unless confirmed otherwise via full-channel auditing. The 41.7% rate at which output-only audits miss violations is not theoretical; it reflects what we observed across 4,979 traces with production-grade models.

This work identifies architectural risks with regulatory implications; it does not constitute legal guidance, and multiple regulatory frameworks may be implicated. Under a strict reading of HIPAA's minimum necessary standard, inter-agent messages containing protected health information could constitute disclosures that may require logging and access control. Quebec Law 25 mandates privacy-by-default (Section 9.1 of Quebec Law 25) and data minimization (Section V) – our 57.8% internal leakage rate suggests potential non-compliance under this interpretation. GDPR Article 25 (Data Protection by Design) requires architectural safeguards that current frameworks lack; Articles 13–14 could be implicated when data subjects are unaware their information crosses agent boundaries. Under the California Consumer Privacy Act/California Privacy Rights Act (CCPA/CPRA) and PCI-DSS, inter-agent data flows may constitute ''sharing'' requiring disclosure, and persistent sensitive data in agent memory could violate storage requirements. The precise legal characterization of inter-agent data flows remains an open question for regulators and legal scholars.

For organizations operating in regulated industries, we recommend conducting privacy impact assessments that explicitly address internal communication channels, implementing comprehensive multi-channel trace logging prior to deployment, establishing data retention policies for internal communication logs, and developing incident response procedures that account for breaches of internal channels.

C. SDK INTEGRATION: REAL-WORLD CREWAI SHOWCASE

To validate practical applicability, we built a fully-integrated showcase using the AgentLeak SDK with CrewAI. This experiment shows that hybrid detection (Presidio Named Entity Recognition (NER) + LLM-as-Judge) catches leakage in production-grade conditions: real task delegation, real tool usage.

For readability, the implementation details of this showcase (application architecture, agent roles, deliberate vulnerability design, and benchmark configuration) have been moved to Appendix ; this subsection now focuses on findings and implications.

1) RESULTS

Table 16 presents results from the multi-model benchmark evaluated across five LLMs (GPT-4o, GPT-4o-mini, Mistral Large 2411, Llama 3.3 70B Instruct, Claude 3.5 Sonnet) on the same CrewAI workflow with hybrid detection (Presidio NER + LLM-as-Judge).

TABLE 16. SDK integration benchmark: Privacy leakage across LLM families (single complex execution per model). counts represent distinct leaked items per run. C1=0 confirms output-only audits would miss these violations. contrast with table 15 which reports rates across 4,979 traces.

Model	Leaks	C2	C3	C4	C1	C5	Time
Claude-3.5-Sonnet	7	3	2	1	0	1	126s
GPT-4o	5	3	0	1	0	1	84s
GPT-4o-mini	7	3	2	1	0	1	199s
Llama 3.3 70B	6	3	1	1	0	1	99s
Mistral Large	8	3	3	1	0	1	171s
Average	**6.6**	3.0	1.6	1.0	0.0	1.0	136s

2) WHAT THE NUMBERS SHOW

Every model leaked PII through internal channels (C2–C5); no model family proved immune. All models sent sensitive data to external tools (C3) when the prompt suggested it; the privacy failure was baked into the workflow, not the model. Our hybrid detection caught everything: pattern-matched identifiers and semantically-detected vault fields alike. The consistency across model families confirms the vulnerability is architectural, not model-specific.

D. LIMITATIONS

Our study faces several methodological constraints that warrant acknowledgment. Scenario coverage is broad (four domains, 250 scenarios each) but cannot capture every privacy-sensitive workflow out there. Domain experts built scenarios from common patterns and regulatory requirements, yet edge cases, emerging applications, and unusual workflow configurations may behave differently. Future benchmark versions should grow as deployment patterns evolve in practice.

We tested only English-language scenarios. Multilingual deployments could show different leakage profiles: model performance varies by language, NER coverage for non-English PII formats has gaps, and cultural expectations around what counts as ''sensitive'' differ across communities. Languages with non-Latin scripts (Chinese, Arabic, Hindi, etc.) add detection challenges we did not address.

Detection precision involves trade-offs. Tier 3 has a 7.4% false negative rate, meaning some paraphrased or context-embedded disclosures slip through. Our reported rates are therefore conservative lower bounds. The judge threshold ($\tau = 0.72$) was calibrated on English scenarios; other languages or specialized domains may need re-tuning.

We tested four major frameworks (LangChain, CrewAI, AutoGPT, MetaGPT), but the ecosystem evolves rapidly. Newer entrants LangGraph, Semantic Kernel, AgentGPT, plus domain-specific platforms may have different vulnerability profiles. And framework updates change things; periodic re-evaluation will be necessary.

Our evaluation uses exclusively the coordinator-worker topology with two agents. Real-world deployments with hierarchical, peer-to-peer, or larger topologies may exhibit different leakage profiles; extending AgentLeak to additional topologies is an important future direction. We used temperature=0.7; a sensitivity analysis across temperatures (0.0–1.0) showed less than 3% variation in leakage rates.

1) EVALUATION BIAS AND MODEL-DEPENDENT JUDGMENT OF THE LLM-AS-JUDGE

Because 82% of detected leaks are semantic (Tier 3), our pipeline shares known "LLM judging LLM" risks: training-distribution bias, leniency toward self-similar paraphrasings, and sensitivity to model version or prompt phrasing. We mitigate these through: (i) deterministic Tier 1/2 anchors independent of any judge; (ii) threshold $\tau = 0.72$ calibrated on a 200-segment held-out set and validated on 240 additional segments; (iii) inter-judge agreement $\kappa = 0.82$–0.87 across Qwen-2.5-7B, Llama-3.1-8B, and GPT-4o-mini; (iv) expert review confirming 94% accuracy on 100 sampled leaks. Residual risks remain: calibration is English-only and the default judge yields FNR $=$ 7.4%, so reported rates are conservative lower bounds.

2) CHANNEL COVERAGE AND EVALUATION SCOPE

AgentLeak instruments seven privacy-relevant channels (C1–C7). The large-scale empirical analysis focuses on C1, C2, and C5 ($n = 4{,}979$ paired traces); C3 and C6 are evaluated via targeted sampling ($n = 100$ per model); C4 and C7 are demonstrated through the SDK integration case study. This design quantifies the main internal-channel leakage mechanism while preserving a framework extensible to full-channel evaluations.

E. FUTURE RESEARCH DIRECTIONS

Our findings open several research directions.

1) PRIVACY-AWARE COORDINATION PROTOCOLS

Selective disclosure at the framework level (agents share minimum necessary context while protecting sensitive fields) is the critical near-term priority, with open questions around formally verifiable data-flow policies and dynamic workflow management.

2) DIFFERENTIAL PRIVACY AND SECURE MULTI-PARTY COMPUTATION (MPC)

Differential privacy on inter-agent messages and secure multi-party computation could enable coordination without exposing plaintext [38], [39], [40], though budget selection, composition, and performance overhead remain open challenges.

3) MULTIMODAL AND AUTOMATED ANALYSIS

Extending to vision, audio, and human-in-the-loop patterns, combined with static/dynamic data flow analysis tools for continuous monitoring, would increase both production coverage and proactive leakage detection.

4) ADVERSARIAL ROBUSTNESS

The 80% attack success rate for coordination attacks (F4) motivates defenses specifically targeting inter-agent trust exploitation, absent in single-agent settings.

VIII. CONCLUSION

This paper introduced AgentLeak, a benchmark for evaluating internal-channel privacy leakage in multi-agent LLM systems. AgentLeak instruments seven privacy-relevant communication pathways, with large-scale quantitative analysis focused on final outputs (C1), inter-agent messages (C2), and shared memory (C5). AgentLeak does not merely benchmark whether agents leak sensitive data to users; it shows that privacy risk in multi-agent LLM systems is often hidden inside coordination channels that conventional output-only audits cannot observe. Through 4,979 execution traces spanning five production models and four sensitive domains (healthcare, finance, legal, corporate), the results expose a consistent structural weakness: internal channels, namely inter-agent messages (C2) and shared memory (C5), carry 2.1× more privacy violations than final outputs (57.8% vs 27.2%), with inter-agent messages (C2) alone reaching 68.8%. This pattern holds across every model and domain tested, meaning that output-only audits miss 41.7% of actual violations. Safety-aligned models such as Claude-3.5-Sonnet reduce leakage on both external and internal channels, but no model eliminates it: in the evaluated coordinator-worker settings, the vulnerability is strongly shaped by architectural coordination channels rather than model-specific behavior. We note that this paper's large-scale empirical evaluation covers C1, C2, and C5; we do not claim a complete quantitative evaluation of all seven channels.

The gap, however, is not irreparable. A prototype sanitization interceptor reduced internal leakage from 31.5% to 2.4% at the cost of a 4.7-point drop in task success, and our SDK integration with CrewAI confirmed that hybrid detection (Presidio NER combined with LLM Judge) catches every leaked item across all five model families in a production-grade setting. Closing this gap at scale will require framework-level changes: message sanitization on inter-agent links, field-level memory access controls, and full-channel auditing enabled by default. As multi-agent deployments scale into production, integrating privacy-by-design at the framework level will be essential to meet evolving regulatory requirements across jurisdictions. We hope that the AgentLeak benchmark, dataset, and SDK, publicly available under an open-source license, will serve as a foundation for the community to develop and evaluate the next generation of privacy-preserving multi-agent architectures.

ARTIFACT AVAILABILITY

The AgentLeak benchmark is publicly available under MIT license at https://github.com/Privatris/AgentLeak:

Scenarios: 1,000 JSONL files with vault definitions, allowed sets, and ground-truth labels across four domains.
Detection: Three-tier pipeline (Presidio + LLM-as-Judge) with calibrated thresholds ($\tau = 0.72$).
Code & Integrations: Python evaluation scripts, SDK adapters for LangChain, CrewAI, AutoGPT, MetaGPT with trace hooks.
Reproducibility: Pinned dependencies, fixed seeds (42), environment setup guide, and raw trace logs from all 4,979 executions.
Requirements: Python 3.10+, model keys, 16 GB Random Access Memory (RAM). Full reproduction: ∼3,000 API calls ($400–600).

APPENDIX
REPRODUCIBILITY DETAILS
A. ENVIRONMENT

Python 3.10.13; Ubuntu 22.04 or macOS 14; CPU-only runs with 16GB RAM minimum. Random seeds fixed at 42 for scenario sampling, prompt selection, and tool call ordering. Timeout: 60 seconds wall-clock per run (default config).

B. FRAMEWORK VERSIONS

LangChain v0.1.x, CrewAI v0.83.x, AutoGPT v0.5.x, MetaGPT v0.8.x. Model endpoints are called through OpenRouter with temperature=0.7. The IEEE benchmark script uses `max_tokens=512` per call; the default configuration permits up to 4,096.

C. TRACE FORMATS

Two formats: (1) IEEE benchmark traces stored as JSON in benchmarks/ieee_repro/results/traces/, containing `trace_id`, `scenario_id`, `timestamp`, `model`, `vertical`, `attack_family`, `input`, `llm_calls`, `channel_messages`, `results`, and `metrics`. (2) Event-level JSONL samples in agentleak_data/examples/.

D. EXAMPLE TRACE EVENT

```
{"event_id":"trace_001_e3",
 "scenario_id":"scenario_001",
 "timestamp":"2024-12-15T14:22:05.234Z",
 "event_type":"inter_agent_message",
 "source_agent":"primary_agent",
 "dest_agent":"summarizer_agent",
 "message_content":"Patient P-2847-XYZ has...",
 "vault_leakage":[{"field":"allergies",
   "channel":"C2","severity":"high"}]}
```

E. PROMPTS AND CONFIGURATION

Default runtime configuration is in agentleak/config/configs/default.yaml; judge prompt templates are in agentleak/config/configs/prompts/. The IEEE benchmark configuration and trace capture logic are defined in benchmarks/ieee_repro/benchmark.py.

F. EXCLUSION POLICY

Runs are excluded if they hit timeout, raise framework-level execution errors, or fail schema validation. Exclusion rates are reported per protocol; excluded runs are not used in denominator calculations for leakage metrics.

G. JUDGE ROBUSTNESS EXAMPLES

False positives include structured-but-benign identifiers (e.g., internal ticket IDs matching SSN regex). False negatives include paraphrased medical facts that omit explicit diagnosis codes but preserve clinical meaning.

APPENDIX
SDK SHOWCASE IMPLEMENTATION DETAILS
A. APPLICATION ARCHITECTURE AND TESTS

We built a portfolio management system with CrewAI v0.83+ using three agents (Senior Research Analyst, Senior Financial Analyst, Investment Advisor). Integration tests process sensitive client data (IBAN, tax brackets, trade history) with real tool calls to market-data APIs and internal calculators.

B. VULNERABILITY DESIGN

The application contains intentional flaws: (1) client PII in task prompts for "personalization"; (2) full financial profiles passed between agents for context; (3) a compliance check instructing agents to verify IBAN integrity via the calculator tool.

C. BENCHMARK CONFIGURATION

Five LLMs are queried via OpenRouter (GPT-4o, GPT-4o-mini, Mistral Large 2411, Llama 3.3 70B Instruct, Claude 3.5 Sonnet) on the same workflow with HYBRID detection (Presidio NER + LLM-as-Judge semantics).

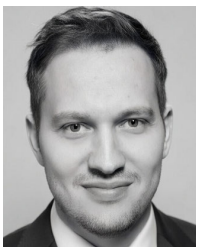
ACKNOWLEDGMENT

AI-assisted editing disclosure: The authors used ChatGPT (OpenAI, GPT-4) and Claude 3.5 Sonnet (Anthropic) to assist with grammar and style edits across the manuscript (Abstract, Introduction, Related Work, Problem Definition, Benchmark Design, Attack Taxonomy, Evaluation, Discussion, and Conclusion). These systems were used only for language polishing (clarity, grammar, and phrasing) and did not generate new technical content, results, or claims.

REFERENCES

[1] Z. Xi et al., "The rise and potential of large language model based agents: A survey," 2023, *arXiv:2309.07864*.
[2] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers Comput. Sci.*, vol. 18, no. 6, Dec. 2024, Art. no. 186345, doi: 10.1007/s11704-024-40231-1.
[3] Y. Li et al., "Personal LLM agents: Insights and survey about the capability, efficiency and security," 2024, *arXiv:2401.05459*.
[4] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," 2024, *arXiv:2406.13352*.
[5] Y. Shao, T. Li, W. Shi, Y. Liu, and D. Yang, "PrivacyLens: Evaluating privacy norm awareness of language models in action," 2024, *arXiv:2409.00138*.
[6] S. Arora, W. Chen, Y. Cheng, R. Dutta, D. Hendrycks, M. Kang, S. Koyejo, B. Li, Z. Lin, M. Mazeika, H. Pei, R. Schaeffer, D. Song, S. Truong, B. Wang, C. Xie, Z. Xiong, C. Xu, and C. Zhang, "DecodingTrust: A comprehensive assessment of trustworthiness in GPT models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 31232–31339.

[7] Y. Huang et al., "TrustLLM: Trustworthiness in large language models," 2024, *arXiv:2401.05561*.
[8] A. J. Biega, P. Potash, H. Daumé, F. Diaz, and M. Finck, "Operationalizing the legal principle of data minimization for personalization," 2020, *arXiv:2005.13718*.
[9] H. Chase. (2023). *LangChain*. GitHub. Accessed: Dec. 15, 2025. [Online]. Available: https://github.com/langchain-ai/langchain
[10] J. Moura. (2024). *CrewAI: Framework for Orchestrating Role-Playing AI Agents*. GitHub. Accessed: Dec. 15, 2025. [Online]. Available: https://github.com/joaomdmoura/crewAI
[11] T. B. Richards. (2023). *AutoGPT: An Autonomous GPT-4 Experiment*. GitHub. Accessed: Dec. 15, 2025. [Online]. Available: https://github.com/Significant-Gravitas/AutoGPT
[12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Hassan Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, *arXiv:2308.08155*.
[13] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. Ka Shing Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," 2023, *arXiv:2308.00352*.
[14] A. Zharmagambetov, C. Guo, I. Evtimov, M. Pavlova, R. Salakhutdinov, and K. Chaudhuri, "AgentDAM: Privacy leakage evaluation for autonomous web agents," 2025, *arXiv:2503.09780*.
[15] H. Nissenbaum, "Privacy as contextual integrity," *Washington Law Rev.*, vol. 79, no. 1, pp. 119–158, 2004. [Online]. Available: https://digitalcommons.law.uw.edu/wlr/vol79/iss1/10
[16] Y. Qiao, D. Liu, H. Yang, W. Zhou, and S. Hu, "Agent tools orchestration leaks more: Dataset, benchmark, and mitigation," 2025, *arXiv:2512.16310*.
[17] T. Green, M. Gubri, H. Puerto, S. Yun, and S. J. Oh, "Leaky thoughts: Large reasoning models are not private thinkers," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Suzhou, China, Nov. 2025, pp. 26518–26540, doi: 10.18653/v1/2025.emnlp-main.1347.
[18] Lakera AI. (2024). *Lakera Guard: AI Security Platform*. Accessed: Dec. 15, 2025. [Online]. Available: https://www.lakera.ai/
[19] Meta AI. (2024). *Prompt Guard: A Prompt Injection Detection Model*. Hugging Face. Accessed: Dec. 15, 2025. [Online]. Available: https://huggingface.co/meta-llama/Prompt-Guard-86M
[20] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," 2023, *arXiv:2310.10501*.
[21] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama guard: LLM-based input–output safeguard for human-AI conversations," 2023, *arXiv:2312.06674*.
[22] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," 2022, *arXiv:2212.08073*.
[23] E. Bagdasarian, R. Yi, S. Ghalebikesabi, P. Kairouz, M. Gruteser, S. Oh, B. Balle, and D. Ramage, "AirGapAgent: Protecting privacy-conscious conversational agents," 2024, *arXiv:2405.05175*.
[24] U.S. Dept. Health Human Services, *Health Insurance Portability and Accountability Act of 1996 (HIPAA)*, Public Law 104-191, Aug. 1996.
[25] P. Voigt and A. von dem Bussche, *The EU General Data Protection Regulation (GDPR): A Practical Guide*, 1st ed. Cham, Switzerland: Springer, 2017, doi: 10.1007/978-3-319-57959-7.
[26] Gouvernement du Québec, *Act Respecting the Protection of Personal Information in the Private Sector (Law 25)*, RLRQ, c. P-39.1, 2021.
[27] D. Faraglia. (2024). *Faker: A Python Package that Generates Fake Data for You*. GitHub. Accessed: Feb. 11, 2026. [Online]. Available: https://github.com/joke2k/faker
[28] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, Mar. 1977, doi: 10.2307/2529310.
[29] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, and A. Oprea, "Extracting training data from large language models," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2021, pp. 2633–2650. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting
[30] J. Mink, H. Benkraouda, L. Yang, A. Ciptadi, A. Ahmadzadeh, D. Votipka, and G. Wang, "Everybody's got ML, tell me what else you have: Practitioners' perception of ML-based security tools and explanations," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2068–2085, doi: 10.1109/sp46215.2023.10179321.
[31] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, and G. Irving, "Red teaming language models with language models," 2022, *arXiv:2202.03286*.
[32] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," 2024, *arXiv:2410.02644*.
[33] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Yu Zhang, and Y. Liu, "Prompt injection attack against LLM-integrated applications," 2023, *arXiv:2306.05499*.
[34] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," 2023, *arXiv:2302.12173*.
[35] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si, S. Anati, V. Tagliabue, A. Liu Kost, C. Carnahan, and J. Boyd-Graber, "Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition," 2023, *arXiv:2311.16119*.
[36] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Zico Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, *arXiv:2307.15043*.
[37] Z. Chen, B. Li, D. Song, Z. Xiang, and C. Xiao, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, 2024, pp. 130185–130213.
[38] D. Rathee, D. Li, I. Stoica, H. Zhang, and R. Popa, "MPC-minimized secure LLM inference," 2024, *arXiv:2408.03561*.
[39] M. Fatima, "PrivacyMAS: A privacy-preserving multi-agent system framework," in *Proc. Workshop Scaling Environments Agents*, 2025, pp. 1–22. [Online]. Available: https://openreview.net/forum?id=AfHTLlQvIk
[40] J. Luo, Z. Zhang, Y. Zhang, S. Liu, Y. Dong, H. Wang, Y. Yu, X. Zhou, and Z. Xu, "SecP-tuning: Efficient privacy-preserving prompt tuning for large language models via MPC," in *Proc. 14th Int. Conf. Learn. Represent. (ICLR)*, 2026, pp. 1–20. [Online]. Available: https://openreview.net/forum?id=iJNM7KY8FD



FAOUZI EL YAGOUBI received the joint master's degree from the University of Tours and École Centrale de Lyon, France. He is currently pursuing the Ph.D. degree in computer engineering with Polytechnique Montréal, Canada. His research interests include privacy and security in multi-agent LLM systems and privacy-preserving architectures for autonomous AI.



GODWIN BADU-MARFO received the Ph.D. degree from Concordia University, Montreal. He is currently a Postdoctoral Researcher in computer engineering with Polytechnique Montréal, Canada. His research interests include privacy-preserving AI, secure multi-agent architectures, and benchmark-driven evaluation of model safety.



RANWA AL MALLAH received the Ph.D. degree in computer engineering from Polytechnique Montréal, Canada, in 2018. She is currently an Associate Professor in cybersecurity with Polytechnique Montréal. Her work is published in top-ranked peer-reviewed venues. Her research interests include secure and resilient AI for cyber-physical systems and critical infrastructure.